Componentes en Lógica de Negocios

Actividad práctica, Unidad 3

Informe de arquitectura de software

Diseño arquitectural del sistema con IA

Eventos Nova

Empresa asignada	Eventos Nova
Equipo	Equipo 16
Integrantes	Omar Tadeo Gasca Deciga (2226708) Osmar Abdi Sanchez Santiago (2226699) Gabriela Alejandra Fernández Briones (2226870)
Curso	Componentes de Lógica de Negocios
Profesor	Miguel Ángel Gómez Marroquín
Fecha de entrega	17/10/26
Estado del documento	Borrador teórico para revisión del equipo. Los supuestos están marcados como pendientes de validación.

Contenido

1. Alcance y preparación del caso (Etapa 1)
2. Selección del patrón de lógica de negocio (Etapa 2)
3. Mapa de dominio (Etapa 3)
4. Service Layer (Etapa 4)
5. Repository y Unit of Work (Etapa 5)
6. Commands y Queries: CQRS básico (Etapa 6)
7. Revisión con SOLID (Etapa 7)
8. Representación de la arquitectura (Etapa 8)
9. Architecture Decision Records
10. Matriz de trazabilidad
11. Validación (Etapa 9)
12. Conclusiones
13. Bitácora de uso de IA
14. Preguntas para la defensa: guía de respuesta

1. Alcance y preparación del caso (Etapa 1)

1.1 Proceso principal elegido

Proceso principal: desde la solicitud del cliente hasta la confirmación de los proveedores y la gestión de cambios del evento. En el BPMN de la Unidad 2 corresponde al tramo *Proporcionar requerimientos → Preparar propuesta → ¿Propuesta aceptada? → Firmar contrato → Registrar anticipo 30% → Coordinar y asignar proveedores → ¿Confirma en < 48 h?*, más la excepción de cambios del BRD (sección 4).

Se eligió este tramo porque concentra las reglas con más lógica del BRD (anticipo 30%, plazo de 48 h, límite de 15 días, versionado) e involucra a los cuatro roles (Cliente, Coordinador, Administración y Proveedor). La ejecución del evento, la liquidación y el cierre quedan documentados como extensiones, no como diseño detallado.

1.2 Casos de uso seleccionados

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Registrar solicitud y generar propuesta V1.0 (flujo normal)	Coordinador (con datos del Cliente)	BN02, BR07, RNF02	Evento registrado y propuesta V1.0 en "Borrador"
CU-02	Aceptar propuesta, registrar contrato y anticipo	Cliente, Administración	BR01, BN07, RNF03	Evento "Confirmado" con anticipo >= 30%
CU-03	Solicitar disponibilidad a proveedores (con excepción de 48 h)	Coordinador, Proveedor, Sistema	BR02, BR06, BN03, BN06, RNF07	Solicitudes en "Confirmado", "Rechazado" o "Expirado" con alerta
CU-04	Modificar evento: aforo, horario o menú (con excepción tardía)	Cliente, Coordinador, Administración	BR03, BR04, BR07, BR10, BN03, BN04, BN05, RNF01, RNF08	Cambio aplicado y nueva versión de propuesta; si faltan menos de 15 días, cargo del 15% pendiente de autorización

1.3 Detalle de cada caso de uso

ID	Información de entrada	Flujo principal	Excepción o rechazo	Aparece en U2
CU-01	Cliente, tipo de evento, fecha, aforo y servicios cotizados	1) El Coordinador registra los datos base. 2) El sistema valida aforo y servicios. 3) Se genera la propuesta V1.0 en "Borrador". 4) El Coordinador la envía.	Aforo menor o igual a cero; propuesta sin servicios	US01, US07; BPMN "Preparar Propuesta"
CU-02	Propuesta enviada, contrato firmado, monto del anticipo	1) El Cliente acepta la propuesta. 2) Se registra el contrato con el monto de la versión vigente. 3) Administración registra el anticipo. 4) Con anticipo acumulado >= 30% el evento pasa a "Confirmado".	Aceptar una propuesta no enviada; anticipo sin contrato; anticipo menor a 30% (el evento sigue "Solicitado"); pago registrado por un rol distinto de Administración	US02, US03; BPMN "Firmar Contrato", "Registrar Anticipo 30%", gateway "¿Propuesta Aceptada?"
CU-03	Evento confirmado, proveedor y servicio a solicitar	1) El Coordinador asigna servicios. 2) Se crea una solicitud "Pendiente" por proveedor y se notifica. 3) El Proveedor confirma o rechaza dentro de 48 h.	Pasan 48 h sin respuesta: la solicitud pasa a "Expirado" y se alerta al Coordinador; rechazo: se busca alternativa en el catálogo Backup	US04, US05, US06; BPMN "Coordinar y Asignar Proveedores", gateway "¿Confirma en < 48h?"
CU-04	Evento confirmado, tipo de cambio, nuevo valor, nuevos servicios cotizados, solicitante	1) Se registra la solicitud de cambio. 2) El sistema evalúa la política: con 15 días o más no hay penalización. 3) El cambio se aplica y se genera una nueva versión de la propuesta. 4) Se notifica a los participantes afectados.	Con menos de 15 días el cambio es una excepción (BR04): se calcula el cargo del 15% y queda pendiente de autorización de Administración; al autorizarse se aplica	US07, US08, US09, US10; BRD sección 4 "Cambio después de la fecha límite"

Los cuatro casos aparecen en las historias de usuario y en el BPMN, por lo que no hay corrección de alcance. Se incluye un flujo normal (CU-01, CU-02) y flujos con excepción (CU-03, CU-04).

1.4 Hallazgos sobre la Unidad 2

La guía exige registrar contradicciones o datos faltantes sin inventar reglas. Las decisiones siguientes son **supuestos provisionales pendientes de validación** con el profesor o el stakeholder.

ID	Qué dicen los documentos	Problema	Decisión provisional de diseño	Estado de Validación
H1	Glosario #14: el 70% restante se paga antes de la ejecución. US14 y BPMN: se liquida después de que el evento finaliza. BR09: no se cierra con saldo pendiente.	No hay un momento único para la liquidación	El modelo no fija el momento: el Evento acepta pagos de liquidación mientras esté "Confirmado" o "Finalizado". El invariante firme es BR09	Validado por el equipo: Se acepta la decisión provisional
H2	US09 dice "faltan >15 días"; US10 dice "faltan <15 días"; BR03 dice "hasta 15 días"	Qué pasa exactamente con 15 días	Prevalece BR03: con 15 días exactos el cambio no se penaliza; con 14 o menos sí	Validado por el equipo: 14 días o menos genera cargo
H3	Glosario #5 y US10: el cargo es automático. BR04: la excepción se revisa para decidir procedencia y costo. Glosario #2: Administración aprueba excepciones	Automático o con revisión	El cargo se calcula automáticamente y el cambio queda "Pendiente de autorización" de Administración	Validado por el equipo: Híbrido, cálculo automático y autorización por Administración
H4	Glosario #5: penalización del 15% a "cualquier modificación"	No se define sobre qué monto se calcula	Se calcula sobre el incremento de costo que produce la modificación (diferencia entre versiones)	Validado por el equipo: Se calcula sobre el incremento del costo
H5	BR08: el pago de último minuto del mismo día va por terminal móvil o transferencia antes del servicio. US13: se suma al saldo "en el momento"	Orden entre cobro y prestación	Un servicio adicional se registra antes de ejecutarse (BR05) y no se marca como prestado sin pago. Queda como extensión fuera de los 4 casos	Validado por el equipo: Se registra como extensión fuera del alcance principal

2. Selección del patrón de lógica de negocio (Etapa 2)

2.1 Tabla comparativa

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Cada caso de uso sería un script largo; las reglas de 30%, 48 h y 15 días se repetirían en varios scripts	Las reglas quedan por tabla (propuesta, pago, proveedor) pero las que cruzan tablas no tienen dueño claro	Cada regla vive en el objeto que la protege (Evento, Propuesta, SolicitudDeProveedor)	BR01, BR02, BR03, BR04, BR09
Cambios esperados	Un cambio de política obliga a editar varios scripts	Un cambio se reparte entre módulos	Una política nueva se agrega como regla sin tocar el resto	BR04 ("revisar procedencia y costo"), P04, OB03
Interacción entre conceptos	La coordinación evento-propuesta-proveedor se resuelve a mano en cada script	Los módulos trabajan sobre conjuntos de filas, sin relaciones entre objetos	Evento, Propuesta y Solicitud se relacionan por identificador y mensajes	BPMN: cuatro carriles y objetos Propuesta V1.0 y Contrato
Estados e invariantes	Las transiciones se validan con condicionales dispersos	Cada tabla valida su propio estado, no los invariantes entre ellas	Las transiciones y los invariantes se protegen dentro de la raíz del Aggregate	US02, US03, US15; Glosario #11 (Pendiente, Confirmado, Rechazado, Expirado)
Historial y trazabilidad	Hay que programar el versionado en cada script	El versionado se resuelve por tabla	Las versiones son parte del modelo y no se sobreescriben	BR07, BR10, RNF01, Glosario #20
Costo inicial y riesgos	Bajo costo y rápido, pero se vuelve difícil de mantener	Costo medio; adecuado si el sistema es sobre todo de datos tabulares	Mayor costo inicial y curva de aprendizaje; riesgo de modelar de más	Equipo de tres integrantes y caso de curso

2.2 Decisión

Patrón principal: Domain Model, con delimitación explícita:

- **Domain Model** para el núcleo con reglas y estados: Evento, Propuesta y SolicitudDeProveedor.
- **Enfoque de script o lectura simple** para consultas e informes (por ejemplo la rentabilidad de US12) y para catálogos sin reglas complejas (Cliente, Proveedor). No se justifica modelar ahí comportamiento de dominio.
- **Table Module** se descarta: el caso no es principalmente de registros tabulares, y las reglas importantes cruzan entidades.

Evidencias de la Unidad 2 que sostienen la decisión:

1. Hay reglas numéricas y temporales que se combinan (30% de anticipo, 48 h de respuesta, 15 días de límite, cargo del 15%): BR01, BR02, BR03, BR04.

2. El negocio se describe como ciclos de vida con estados: propuesta (Borrador, Aceptada), evento (Confirmado, Liquidado, Histórico) y proveedor (Pendiente, Confirmado, Rechazado, Expirado): US02, US03, US15, Glosario #11.

3. Hay invariantes que no deben romperse aunque cambie la pantalla o el canal: no cerrar con saldo pendiente (BR09, RNF06) y no sobreescribir versiones (BR07, Glosario #20).

4. Los cambios producen consecuencias comerciales rastreables (P04, OB03, BR10).

Consecuencias aceptadas: más estructura inicial que un CRUD, necesidad de mapear los objetos a la base de datos y riesgo de agregados demasiado grandes. Se mitiga con Aggregates pequeños y con la separación de lecturas (CQRS básico, sección 6).

3. Mapa de dominio (Etapa 3)

3.1 Lenguaje ubicuo

Los nombres del diseño usan los términos del Glosario de Dominio de la Unidad 2.

Término (Glosario)	Elemento del diseño	Tipo
#9 Cotización (Propuesta)	Propuesta	Aggregate (raíz)
#20 Versionado de Documentos	NumeroDeVersion (V1.0, V1.1) y VersionDePropuesta	Value Object y entidad inmutable
#6 Cliente	Cliente	Entidad (catálogo)
#7 Contrato	Contrato firmado con el monto de la propuesta aceptada	Parte del Evento
#3 Anticipo (30%) y #14 Liquidación	Pago de tipo Anticipo o Liquidación	Value Object dentro del Evento
#5 Cargo por Urgencia y #12 Fecha Límite (15 días)	PoliticaDeCambios con ReglaDeCargoPorUrgencia	Domain Service
#11 Estado de Proveedor	EstadoDeProveedor	Value Object (enumeración)
#4 Backup de Respuesta Rápida y #16 Proveedor Categoría A	Atributos del Proveedor (es_backup, categoria)	Atributos de entidad
#19 Tipo de Evento	TipoDeEvento (Boda, Graduación, Empresarial)	Value Object
#1 Acta de Cierre	Registro de cierre del Evento	Hecho del dominio (extensión)
#18 Servicio Adicional	ServicioAdicional dentro del Evento	Entidad (extensión)
#17 Rentabilidad	Consulta de lectura, no entidad	Query
#2 Administración, #8 Coordinador	Responsable (rol y nombre)	Value Object
#13 Gateway de Aprobación	No es concepto de dominio: es notación del proceso; se refleja en estados e invariantes	Fuera del modelo
#15 Plataforma Centralizada	El sistema completo	Fuera del modelo

Términos nuevos o corregidos para el glosario: *Solicitud de Cambio* (registro de una modificación con responsable, fecha, condición anterior y nueva, según BR10), *Cambio Pendiente de Autorización* (cambio tardío a la espera de Administración) y *Solicitud de Proveedor* (petición de disponibilidad con plazo de 48 h).

3.2 Entidades y Value Objects

Tipo	Elementos	Por qué
Entidades	Evento, Propuesta, VersionDePropuesta, SolicitudDeProveedor, SolicitudDeCambio, Proveedor, Cliente	Tienen identidad y ciclo de vida; deben distinguirse aunque cambien sus datos
Value Objects	Dinero, Aforo, FechaDeEvento, NumeroDeVersion, LineaDeServicio, Pago, Responsable, TipoDeEvento, EstadoDeProveedor	Se definen por sus valores y reglas (por ejemplo, el aforo no puede ser cero)
No se modelan como entidad	Tablas de apoyo, pantallas, el gateway del BPMN	No tienen comportamiento ni identidad propia en el negocio

3.3 Fichas de Aggregates

Aggregate	Raíz	Responsabilidades	Invariantes	Eventos posibles
Propuesta	Propuesta (contiene VersionDePropuesta y LineaDeServicio)	Guardar el historial de versiones y su estado	INV-P1: las versiones son inmutables y nunca se sobreescriben (BR07). INV-P2: hay exactamente una versión vigente, la más reciente. INV-P3: solo se acepta una propuesta enviada. INV-P4: toda versión tiene al menos un servicio	PropuestaGenerada, NuevaVersionDePropuestaGenerada, PropuestaAceptada

Aggregate	Raíz	Responsabilidades	Invariantes	Eventos posibles
Evento	Evento (contiene Contrato, Pago y SolicitudDeCambio)	Controlar el ciclo de vida, los pagos y los cambios	INV-E1: solo está "Confirmado" con contrato firmado y pagos >= 30% del monto total (BR01). INV-E2: el anticipo solo lo registra Administración y solo con contrato (RNF03). INV-E3: un cambio con menos de 15 días queda pendiente de autorización con cargo del 15% (BR04). INV-E4: todo cambio guarda responsable, fecha, condición anterior y nueva (BR10). INV-E5: no se cierra con saldo pendiente (BR09, RNF06). INV-E6: el saldo es monto total más cargos aplicados menos pagos; no existe operación de reembolso del anticipo (BR01)	ContratoFirmado, AnticipoRegistrado, EventoConfirmado, CambioPendienteDeAutorizacion, CambioAplicado, CargoPorUrgenciaAplicado, EventoLiquidado, EventoCerrado
SolicitudDeProveedor	SolicitudDeProveedor	Controlar la respuesta del proveedor y su plazo	INV-S1: solo se confirma o rechaza mientras esté "Pendiente" y dentro de 48 h (BR02). INV-S2: pasadas las 48 h sin respuesta pasa a "Expirado" y no regresa a "Pendiente". INV-S3: una solicitud de sustitución referencia a la anterior (BR06)	SolicitudAProveedorEnviada, ProveedorConfirmo, ProveedorRechazo, SolicitudDeProveedorExpirada
Proveedor y Cliente	Proveedor, Cliente	Catálogos con datos de referencia	Sin invariantes complejos (se tratan como CRUD simple)	Ninguno

Reglas de frontera: los Aggregates se referencian entre sí por identificador (evento_id, proveedor_id), nunca por objeto, y cada transacción modifica un Aggregate salvo las excepciones documentadas en la sección 5.

3.4 Domain Services y Domain Events

Elemento	Descripción	Justificación
Domain Service: PoliticaDeCambios	Evalúa, con la fecha del evento, la fecha del cambio y el incremento de costo, si hay cargo y si se requiere autorización. Compone reglas (ReglaDeCambio); hoy existe ReglaDeCargoPorUrgencia	No pertenece a un solo Aggregate, depende de datos de varios y es una política que puede cambiar (BR03, BR04)
Domain Events	Propuesta: PropuestaGenerada, NuevaVersionDePropuestaGenerada, PropuestaAceptada. Evento: ContratoFirmado, AnticipoRegistrado, EventoConfirmado, CambioPendienteDeAutorizacion, CambioAplicado, CargoPorUrgenciaAplicado, EventoLiquidado, EventoCerrado. Solicitud: SolicitudAProveedorEnviada, ProveedorConfirmo, ProveedorRechazo, SolicitudDeProveedorExpirada	Son hechos ya ocurridos que interesan a otros componentes: notificar proveedores (US04, US08), alertar al Coordinador (US06) y avisar a Administración (BR04)

3.5 Relaciones y límites del mapa

- **Evento** referencia al Cliente por identificador. Contiene sus Pagos y Solicitudes de Cambio (composición).
- **Propuesta** referencia al Evento por identificador. Contiene sus Versiones, y estas sus Líneas de Servicio.
- **SolicitudDeProveedor** referencia al Evento y al Proveedor por identificador.
- **PoliticaDeCambios** es consultada por el Evento al solicitar un cambio; no guarda estado.
- Los **Domain Events** salen de los Aggregates y los consumen manejadores de la capa de aplicación.

3.6 Decisiones dudosas y descartadas

Duda	Decisión	Razón	Alternativa descartada
¿Los pagos son un Aggregate aparte (CuentaDelEvento)?	Quedan dentro del Evento	INV-E1 e INV-E5 exigen consistencia inmediata entre estado, contrato y pagos	Cuenta separada: obligaría a coordinar por eventos y permitiría estados contradictorios
¿La SolicitudDeProveedor vive dentro del Evento?	Aggregate separado	Cada proveedor responde en paralelo y con su propio plazo; dentro del Evento bloquearía el Aggregate completo	Solicitudes como lista dentro de Evento
¿La Propuesta es parte del Evento?	Aggregate separado	Existe antes de que el evento esté confirmado y su historial de versiones crece	Versiones dentro del Evento: Aggregate demasiado grande

Duda	Decisión	Razón	Alternativa descartada
¿Backup y Categoría A son subtipos de Proveedor?	Son atributos	Son clasificaciones que pueden combinarse, no comportamientos distintos (Glosario #4 y #16)	Subclases ProveedorBackup y ProveedorCategoriaA

4. Service Layer (Etapa 4)

4.1 Application Services

Cada método expresa una intención completa del negocio y recibe un Command (sección 6).

Servicio	Método o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
CotizacionService	registrar_solicitud_de_evento (CU-01)	Cliente, tipo, fecha, aforo, servicios cotizados, coordinador	UnidadDeTrabajo, RepositorioDeEventos, RepositorioDePropuestas, Reloj	Identificadores del evento y de la propuesta V1.0	Aforo mayor que cero y al menos un servicio (Propuesta); estado inicial "Borrador"
CotizacionService	generar_nueva_version_de_propuesta (CU-01)	Propuesta, servicios, aforo, motivo	UnidadDeTrabajo, RepositorioDePropuestas, Reloj	Nueva versión (V1.1, V1.2...)	Inmutabilidad de versiones (Propuesta)
ContratacionService	aceptar_propuesta (CU-02)	Identificador de propuesta	UnidadDeTrabajo, RepositorioDePropuestas	Propuesta "Aceptada"	Solo se acepta una propuesta enviada (Propuesta)
ContratacionService	registrar_contrato_firmado (CU-02)	Identificador de evento	UnidadDeTrabajo, RepositorioDeEventos, RepositorioDePropuestas	Evento con contrato y monto	La propuesta debe estar aceptada (Propuesta entrega su monto aceptado); monto mayor que cero (Evento)
ContratacionService	registrar_anticipo (CU-02)	Evento, monto, administrador	UnidadDeTrabajo, RepositorioDeEventos, Reloj	Estado y saldo del evento	30% mínimo, contrato firmado, rol Administración, sin reembolso (Evento)
ProveedoresService	solicitar_disponibilidad (CU-03)	Evento, lista de proveedor y servicio	UnidadDeTrabajo, RepositorioDeEventos, RepositorioDeSolicitudesDeProveedor, RepositorioDeProveedores, Reloj	Solicitudes "Pendiente"	Solo un evento confirmado coordina proveedores (Evento)
ProveedoresService	responder_solicitud (CU-03)	Solicitud, acepta o rechaza	UnidadDeTrabajo, RepositorioDeSolicitudesDeProveedor, Reloj	Solicitud "Confirmado" o "Rechazado"	Plazo de 48 h y estado "Pendiente" (SolicitudDeProveedor)
ProveedoresService	expirar_solicitudes_vencidas (CU-03, disparado por temporizador)	Ninguna	UnidadDeTrabajo, RepositorioDeSolicitudesDeProveedor, Reloj	Solicitudes "Expirado" y alertas	Cuándo vence una solicitud (SolicitudDeProveedor)
CambiosService	solicitar_cambio_de_evento (CU-04)	Evento, tipo, nuevo valor, nuevos servicios cotizados, solicitante	UnidadDeTrabajo, RepositorioDeEventos, RepositorioDePropuestas, PoliticaDeCambios, Reloj	Cambio aplicado o pendiente de autorización	Límite de 15 días, cargo del 15% y trazabilidad (Evento y PoliticaDeCambios)
CambiosService	autorizar_cambio (CU-04)	Evento, cambio, administrador	UnidadDeTrabajo, RepositorioDeEventos, RepositorioDePropuestas	Cambio aplicado y nueva versión	Solo Administración autoriza (Evento)

4.2 Qué hace y qué no hace un servicio

- **Sí hace:** abrir la Unidad de Trabajo, recuperar los Aggregates, llamar al método del dominio que expresa la intención, confirmar la transacción y publicar los Domain Events.
- **No hace:** decidir si procede el 30%, si pasaron 48 h o si faltan menos de 15 días. Regla práctica: si un condicional contiene un número del negocio (30%, 48, 15) o un estado, va al dominio.
- **Único cálculo en el servicio:** el incremento de costo de un cambio (total de la nueva cotización menos el total vigente). Es una operación de coordinación de datos entre dos Aggregates, no una regla del negocio.

4.3 Revisión obligatoria

Pregunta	Resultado
¿El nombre del método expresa una acción del negocio?	Sí: registrar_anticipo, autorizar_cambio, expirar_solicitudes_vencidas
¿Coordina o decide?	Coordina; las decisiones están en Evento, Propuesta, SolicitudDeProveedor y PoliticaDeCambios
¿Depende de interfaces?	Sí: solo de puertos (repositorios, UnidadDeTrabajo, Reloj, notificadores)
¿Hay inicio y fin transaccional claro?	Sí: cada método abre y cierra una Unidad de Trabajo (sección 5.2)
¿Los errores se explican en términos del negocio?	Sí: el dominio lanza errores como "No se puede registrar el anticipo sin contrato firmado" o "Un evento no puede cerrarse con saldo pendiente"

5. Repository y Unit of Work (Etapa 5)

5.1 Interfaces de Repository (capa de aplicación)

Los métodos usan lenguaje del dominio y no exponen SQL.

Repository	Necesidad del dominio	Métodos propuestos	Objeto que recupera o guarda
RepositorioDeEventos	Recuperar el Evento completo con pagos y cambios	obtener(evento_id); agregar(evento)	Evento (Aggregate completo)
RepositorioDePropuestas	Recuperar el historial de versiones de un evento	obtener(propuesta_id); obtener_por_evento(evento_id); agregar(propuesta)	Propuesta con sus versiones
RepositorioDeSolicitudesDeProveedor	Encontrar solicitudes de un evento y las que ya vencieron	obtener(solicitud_id); listar_por_evento(evento_id); listar_pendientes_vencidas_al(ahora); agregar(solicitud)	SolicitudDeProveedor
RepositorioDeProveedores	Encontrar proveedores y alternativas Backup	obtener(proveedor_id); listar_backup_para_servicio(servicio)	Proveedor

Los cambios hechos a un Aggregate recuperado se persisten al confirmar la Unidad de Trabajo; por eso no hay un método "guardar" por separado.

5.2 Unit of Work y alcance transaccional

Unidad de Trabajo: inicia al entrar al método del servicio, agrupa todos los cambios de los Aggregates recuperados, y los confirma juntos con *commit*. Si ocurre un error de dominio o de infraestructura, o si no se llega al *commit*, se hace *rollback* y no queda ningún cambio. Los Domain Events se publican **después** del *commit*, para no notificar algo que no se guardó.

Caso de uso	Aggregates en la transacción	Qué incluye	Ante un error
CU-01 registrar solicitud	Evento y Propuesta (alta inicial)	Crea el evento y su propuesta V1.0 juntos	No se crea ninguno
CU-02 aceptar propuesta	Propuesta	Estado "Aceptada"	No cambia el estado
CU-02 registrar contrato	Evento (lee la Propuesta)	Contrato y monto	Sin contrato
CU-02 registrar anticipo	Evento	Pago y, si procede, estado "Confirmado"	Sin pago y sin confirmación
CU-03 solicitar disponibilidad	Solicitudes de Proveedor	Todas las solicitudes del lote	No se envía ninguna
CU-03 responder o expirar	Una solicitud (o cada una por separado en la expiración)	Cambio de estado	La solicitud sigue "Pendiente"
CU-04 solicitar o autorizar cambio	Evento y Propuesta	Cambio, cargo y nueva versión juntos	Ni cambio ni versión (evita versiones contradictorias, RNF08)

5.3 Posible implementación en infraestructura

Se describe aparte del dominio y no forma parte del modelo.

- **Base relacional con ORM:** el Evento se mapea a las tablas evento, pago y solicitud_de_cambio; la Propuesta a propuesta, version_de_propuesta y linea_de_servicio; además solicitud_de_proveedor, proveedor y cliente.
- **Concurrencia:** columna de versión en cada tabla raíz (bloqueo optimista) para detectar dos personas modificando el mismo Aggregate.
- **Pruebas:** una implementación en memoria de los mismos puertos permite probar las reglas sin base de datos.
- **Notificaciones:** adaptadores para correo o portal detrás de los puertos de notificación; el dominio no los conoce.
- **Lecturas:** consultas SQL directas en lectores de la capa de infraestructura, no en el modelo.

6. Commands y Queries: CQRS básico (Etapa 6)

6.1 Operaciones

Cada Command lo atiende un método de Application Service (sin bus de mensajes ni bases separadas, ADR-004). Cada Query la atiende un manejador de lectura que no recupera Aggregates.

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificación
Command	RegistrarSolicitudDeEvento	Cliente, tipo, fecha, aforo, servicios, coordinador	CotizacionService.registrar_solicitud_de_evento	Identificadores de evento y propuesta	Cambia estado; origina CU-01 (US01)
Command	GenerarNuevaVersionDePropuesta	Propuesta, servicios, aforo, motivo	CotizacionService.generar_nueva_version_de_propuesta	Número de versión	Protege el historial (US07, BR07)
Command	AceptarPropuesta	Propuesta	ContratacionService.aceptar_propuesta	Estado "Aceptada"	Inicia el contrato (US02)
Command	RegistrarAnticipo	Evento, monto, administrador	ContratacionService.registrar_anticipo	Saldo y estado del evento	Valida el 30% y confirma el evento (US03, BR01)
Command	SolicitarDisponibilidadAProveedores	Evento, proveedores y servicios	ProveedoresService.solicitar_disponibilidad	Solicitudes pendientes	Dispara notificaciones (US04)
Command	SolicitarCambioDeEvento	Evento, tipo, valor, servicios, solicitante	CambiosService.solicitar_cambio_de_evento	Cambio aplicado o pendiente	Aplica la política de 15 días (US09, US10)
Query	ConsultarPropuestaVigente	Evento	Manejador de lectura de propuestas	PropuestaVigenteVM: versión, estado, aforo, servicios, total	Todos ven la versión vigente (RNF02, BN02, BR07)
Query	ConsultarSolicitudesDeProveedor	Evento	Manejador de lectura de solicitudes	SolicitudProveedorVM: proveedor, servicio, estado, vence_el, horas restantes	Seguimiento de plazos de 48 h (BN06, OB02)
Query	ConsultarSaldoDelEvento	Evento	Manejador de lectura de saldo	SaldoDelEventoVM: monto total, pagado, saldo, anticipo requerido y cubierto	Control de pagos (BN07)
Query	ConsultarRentabilidadPorEvento	Evento	Manejador de lectura de rentabilidad	RentabilidadVM: ingresos, costos de proveedores y margen	US12 y BN05; es solo lectura y no necesita el modelo de dominio

6.2 Dependencias de los Handlers

- **Commands:** UnidadDeTrabajo, repositorios del Aggregate que modifican, Reloj, y PoliticaDeCambios en el caso de cambios.
- **Queries:** únicamente puertos de lectura pequeños (uno por consulta); no usan repositorios de escritura ni modifican estado.
- Los View Models de lectura se diseñan para la pantalla (por ejemplo, horas restantes de una solicitud) y **no** se reutilizan como modelo de escritura.

6.3 Qué no necesita esta separación

El alta y la consulta de **catálogos** (Cliente y Proveedor) son operaciones simples de datos. Se manejan como CRUD directo, sin Commands, Queries ni eventos, porque no tienen reglas que proteger.

7. Revisión con SOLID (Etapa 7)

Hallazgos de la revisión del **borrador inicial** del diseño y los cambios incorporados en esta versión.

#	Principio	Evidencia en el borrador	Impacto	Cambio realizado
1	S	El servicio de cambios calculaba los días restantes y el 15% (regla de BR03, BR04 y US10)	Regla del negocio dispersa, repetida en US09 y US10 y difícil de probar	La regla pasa a PoliticaDeCambios (dominio); el servicio solo orquesta
2	O	Los cargos se resolvían con condicionales por tipo de cambio	Cada regla nueva obligaba a editar código existente (BR04 "revisar procedencia y costo")	ReglaDeCambio como variante; PoliticaDeCambios compone reglas sin editarlas
3	L	Proveedor tenía subclases ProveedorCategoriaA y ProveedorBackup	Un proveedor puede ser A y Backup a la vez; los subtipos no son sustituibles	Se reemplazan por atributos categoria y es_backup (Glosario #4 y #16)
4	I	Una sola interfaz de notificaciones con avisos a cliente, proveedor, coordinador y administración	Cada manejador dependía de métodos que no usa y las pruebas necesitaban dobles enormes (RNF07, US06, US08)	Tres interfaces pequeñas: NotificadorAProveedores, AlertasACoordinacion y AlertasAAdministracion
5	D	Los servicios consultaban la hora del sistema y los dominios llamaban a la notificación	Imposible probar 48 h y 15 días sin esperar; el dominio dependía de infraestructura	Reloj y repositorios como puertos; el dominio recibe fecha y hora como parámetro y publica Domain Events
6	S	Un repositorio con consultas de reporte (rentabilidad) mezcladas con las de escritura	Dos razones de cambio en una interfaz	Las lecturas pasan a manejadores de Query separados (sección 6)
7	Confirmación	Los Aggregates se referencian por identificador y no por objeto	Reduce acoplamiento y bloqueos entre Evento, Propuesta y Solicitud	Se confirma como decisión de diseño

8. Representación de la arquitectura (Etapa 8)

Esta sección es la **especificación textual** de los diagramas, lista para dibujarse en draw.io, PlantUML o Mermaid. Los archivos fuente y exportaciones son un entregable aparte.

8.1 Diagrama de clases: capas, clases y relaciones

Capa	Elementos	Relaciones principales
Dominio	Aggregates: Evento, Propuesta, SolicitudDeProveedor. Entidades: VersionDePropuesta, SolicitudDeCambio, Proveedor, Cliente. Value Objects: Dinero, Aforo, FechaDeEvento, NumeroDeVersion, LineaDeServicio, Pago, Responsable. Domain Service: PoliticaDeCambios, ReglaDeCambio (interfaz), ReglaDeCargoPorUrgencia. Domain Events	Propuesta contiene (composición) VersionDePropuesta, y esta LineaDeServicio. Evento contiene Pago y SolicitudDeCambio. Evento usa PoliticaDeCambios. ReglaDeCargoPorUrgencia realiza ReglaDeCambio. Referencias por identificador: Propuesta a Evento, Evento a Cliente, SolicitudDeProveedor a Evento y Proveedor
Aplicación	Application Services: CotizacionService, ContratacionService, ProveedoresService, CambiosService. Commands y Queries con View Models. Manejadores de lectura. Despachador y manejadores de Domain Events. Puertos: UnidadDeTrabajo, los cuatro repositorios, Reloj, NotificadorAProveedores, AlertasACoordinacion, AlertasAAdministracion	Los servicios dependen de los puertos, no de implementaciones. Los servicios invocan a los Aggregates y publican sus eventos. Los manejadores de eventos usan los puertos de notificación
Infraestructura	UnidadDeTrabajo y repositorios (ORM o en memoria), adaptadores de notificación (correo o portal), Reloj del sistema, lectores de consulta	Cada adaptador realiza (implementa) un puerto de la capa de aplicación

Dirección de dependencias: Infraestructura depende de Aplicación, y Aplicación depende de Dominio. El Dominio no depende de ninguna.

8.2 Diagrama de secuencia 1 (flujo principal): registrar anticipo y confirmar el evento (CU-02)

Participantes: Administración, ContratacionService, UnidadDeTrabajo, RepositorioDeEventos, Evento, DespachadorDeEventos, AlertasACoordinacion.

1. Administración envía RegistrarAnticipo a ContratacionService.

2. El servicio abre la UnidadDeTrabajo y pide al RepositorioDeEventos el Evento.

3. El servicio llama a Evento.registrar_anticipo(monto, fecha, responsable).

4. **Alternativa, error de regla:** si el rol no es Administración o no hay contrato firmado, el Evento lanza un error de dominio; el servicio hace *rollback* y devuelve el mensaje de negocio.

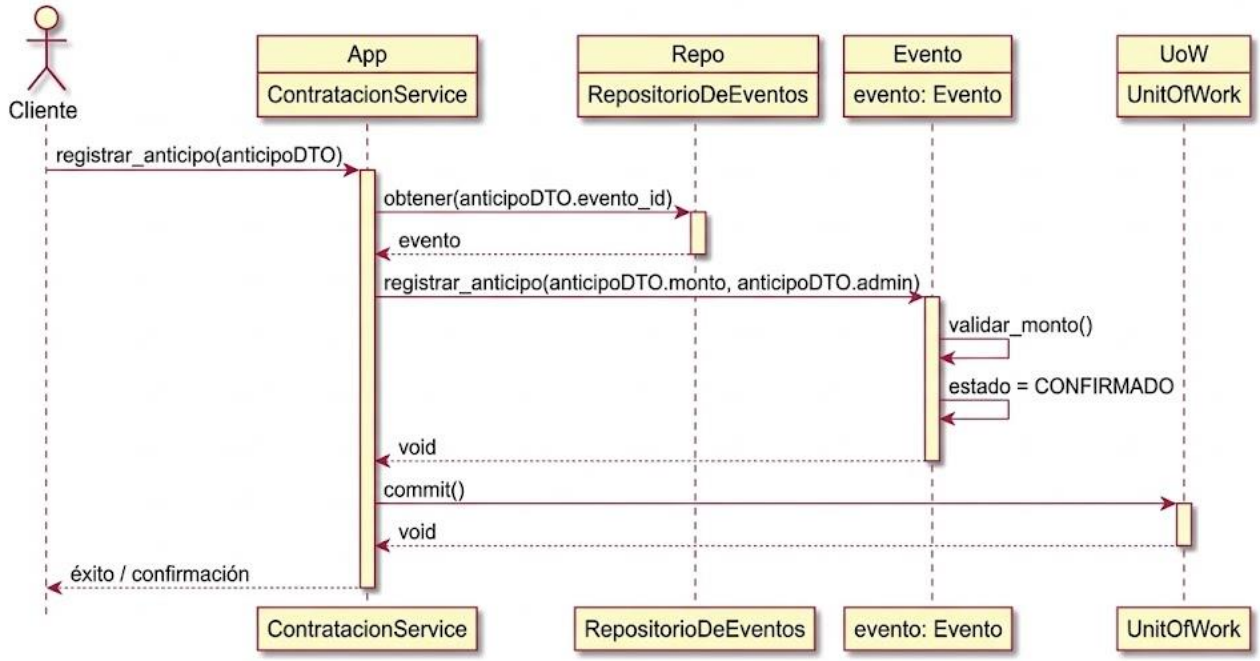
5. **Alternativa, flujo válido:** el Evento registra el Pago de tipo Anticipo; si lo pagado alcanza el 30% del monto total, pasa a "Confirmado" y registra AnticipoRegistrado y EventoConfirmado.

6. El servicio confirma (*commit*) la UnidadDeTrabajo.

7. El servicio publica los eventos en el DespachadorDeEventos, que avisa al Coordinador mediante AlertasACoordinacion que debe asignar proveedores.

8. El servicio devuelve el estado y el saldo a Administración.

8.2 Diagrama de secuencia 1 (flujo principal): registrar anticipo y confirmar el evento (CU-02)



8.3 Diagrama de secuencia 2 (regla y alternativa): cambio tardío de evento (CU-04)

Participantes: Coordinador, CambiosService, UnidadDeTrabajo, RepositorioDeEventos, RepositorioDePropuestas, Evento, PoliticaDeCambios, Propuesta, DespachadorDeEventos, notificadores.

1. El Coordinador envía SolicitarCambioDeEvento a CambiosService.

2. El servicio abre la UnidadDeTrabajo y recupera el Evento y su Propuesta.

3. El servicio calcula el incremento (nuevo total menos total vigente).

4. El servicio llama a Evento.solicitar_cambio(cambio, política, hoy).

5. El Evento consulta a PoliticaDeCambios, que responde cargo y si requiere autorización.

6. **Alternativa, faltan 15 días o más:** el cambio se aplica sin cargo y el servicio pide a la Propuesta una nueva versión.

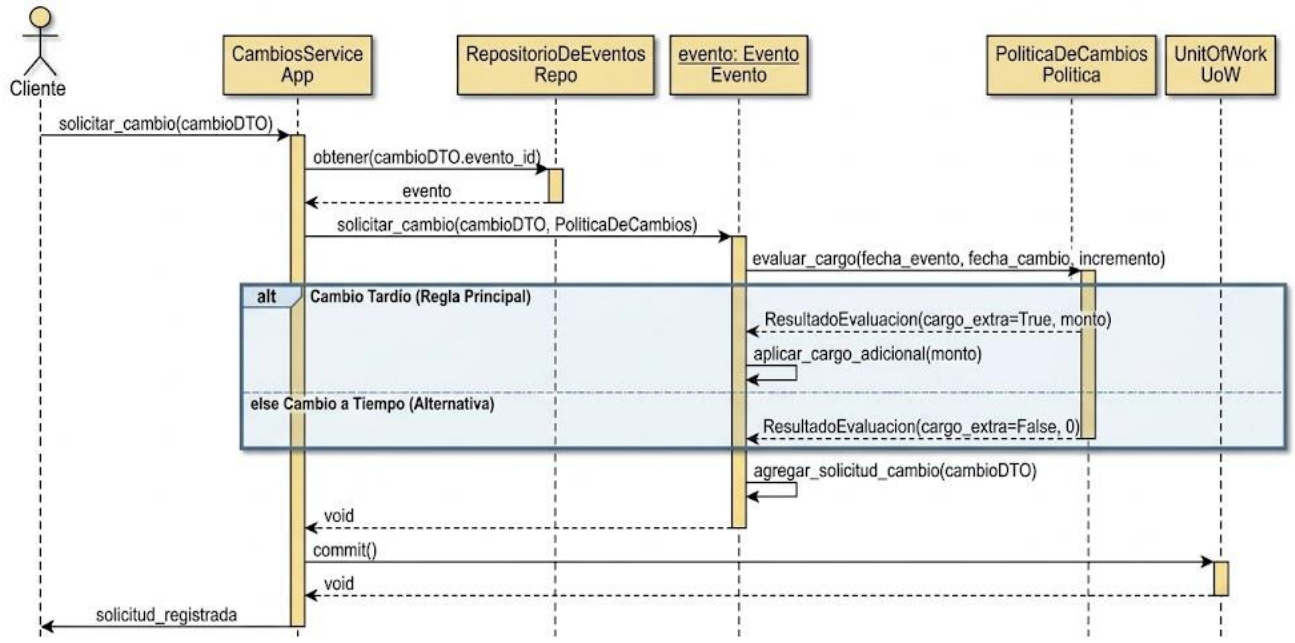
7. **Alternativa, faltan menos de 15 días (excepción, BR04):** el cambio queda "Pendiente de autorización" con cargo del 15% y la Propuesta no cambia hasta que Administración autorice.

8. El servicio confirma (*commit*) una sola transacción con Evento y Propuesta (ADR-003).

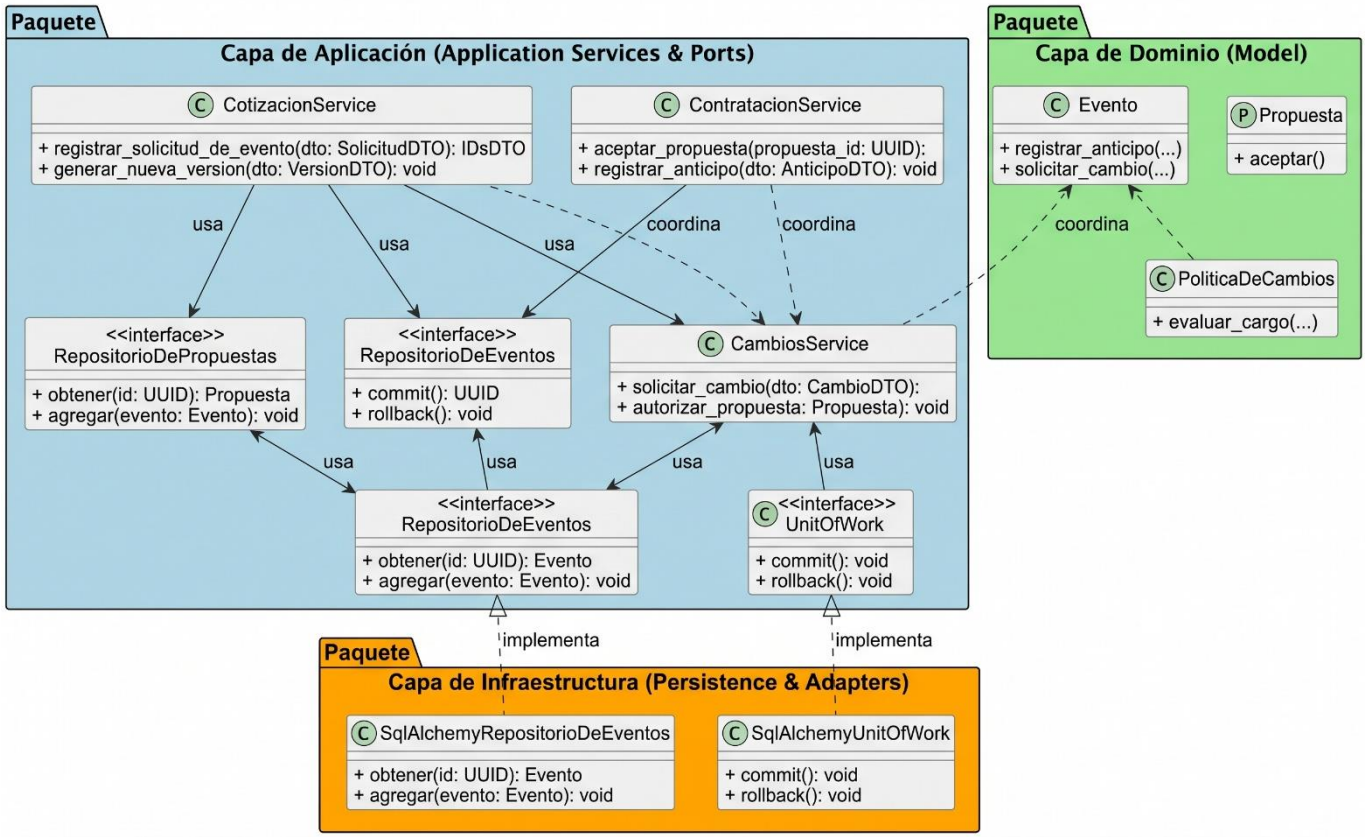
9. El servicio publica los eventos: CambioAplicado notifica a los proveedores afectados; CambioPendienteDeAutorizacion alerta a Administración.

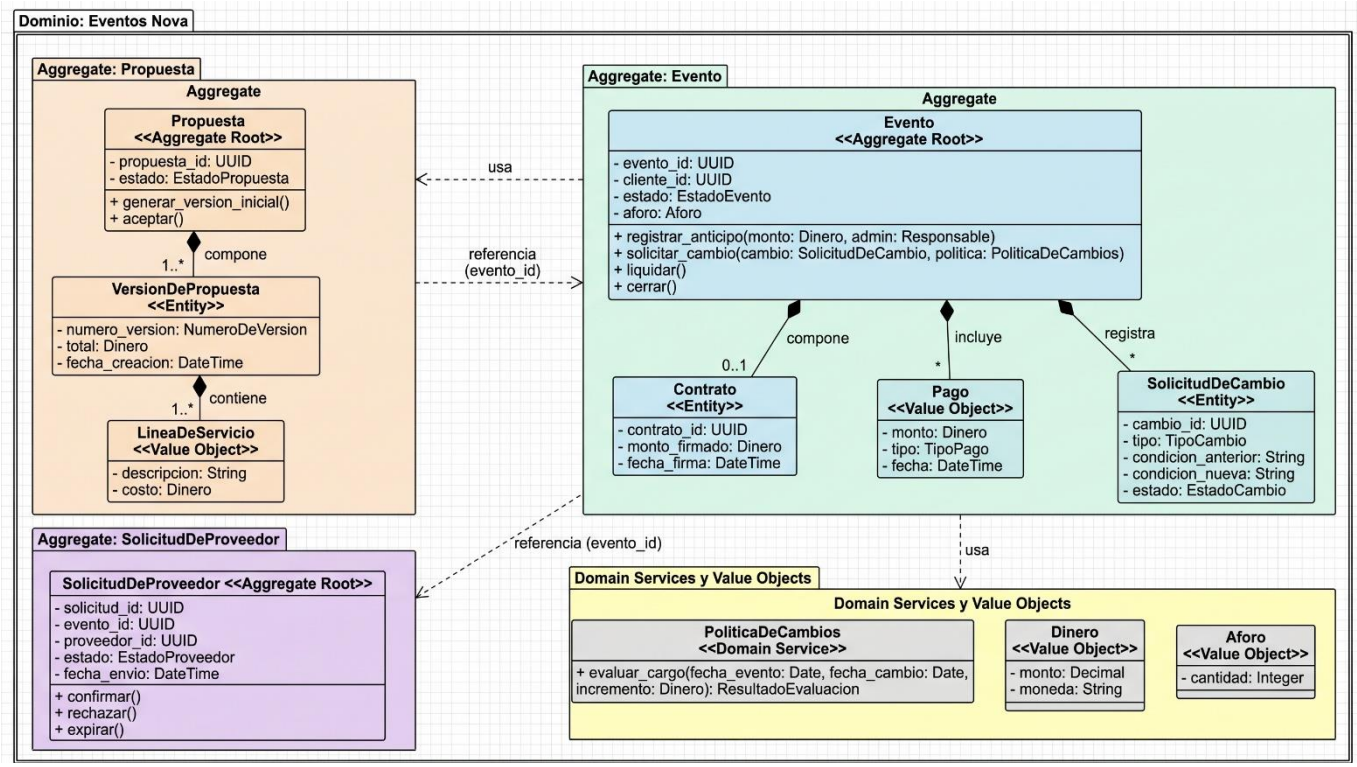
10. El servicio devuelve el resultado al Coordinador.

8.3 Diagrama de secuencia 2 (regla y alternativa): cambio tardío de evento (CU-04)



Consistencia con el diagrama de clases: todos los participantes de las secuencias aparecen como clases o puertos del diagrama de clases.




Dominio: Eventos Nova
Aggregate: Propuesta
Aggregate
Propuesta
<<Aggregate Root>>
- propuesta_id: UUID
- estado: EstadoPropuesta
+ generar_version_inicial()
+ aceptar()
compone
1..*
VersionDePropuesta
<<Entity>>
- numero_version: NumeroDeVersion
- total: Dinero
- fecha_creacion: DateTime
contiene
1..*
LineaDeServicio
<<Value Object>>
- descripcion: String
- costo: Dinero
usa
referencia
(evento_id)
Aggregate: Evento
Aggregate
Evento
<<Aggregate Root>>
- evento_id: UUID
- cliente_id: UUID
- estado: EstadoEvento
- aforo: Aforo
+ registrar_anticipo(monto: Dinero, admin: Responsable)
+ solicitar_cambio(cambio: SolicitudDeCambio, politica: PoliticaDeCambios)
+ liquidar()
+ cerrar()
compone
0..1
incluye
*
registra
*
Contrato
<<Entity>>
- contrato_id: UUID
- monto_firmado: Dinero
- fecha_firma: DateTime
Pago
<<Value Object>>
- monto: Dinero
- tipo: TipoPago
- fecha: DateTime
SolicitudDeCambio
<<Entity>>
- cambio_id: UUID
- tipo: TipoCambio
- condicion_anterior: String
- condicion_nueva: String
- estado: EstadoCambio
Aggregate: SolicitudDeProveedor
SolicitudDeProveedor <<Aggregate Root>>
- solicitud_id: UUID
- evento_id: UUID
- proveedor_id: UUID
- estado: EstadoProveedor
- fecha_envio: DateTime
+ confirmar()
+ rechazar()
+ expirar()
referencia (evento_id)
usa
Domain Services y Value Objects
Domain Services y Value Objects
PoliticaDeCambios
<<Domain Service>>
+ evaluar_cargo(fecha_evento: Date, fecha_cambio: Date, incremento: Dinero): ResultadoEvaluacion
Dinero
<<Value Object>>
- monto: Decimal
- moneda: String
Aforo
<<Value Object>>
- cantidad: Integer

9. Architecture Decision Records

ADR-001: Domain Model como patrón principal de lógica de negocio

Título	Domain Model como patrón principal
Estado	Propuesta
Contexto	Las reglas de 30%, 48 h y 15 días se combinan con estados y versiones (BR01 a BR04, BR07, BR09, Glosario #11 y #20)
Alternativas	Transaction Script: simple, pero duplica reglas en cada caso de uso. Table Module: ordena por tabla, pero las reglas importantes cruzan tablas
Decisión	Domain Model para Evento, Propuesta y SolicitudDeProveedor; lecturas y catálogos con enfoque simple
Consecuencias	Beneficios: reglas protegidas en un solo lugar y fáciles de probar. Costos: más estructura inicial y mapeo a la base de datos. Riesgo: Aggregates grandes. Trabajo posterior: validar los supuestos H1 a H5
Trazabilidad	BR01, BR02, BR03, BR04, BR07, BR09, BN02, US02, US03, US05, US10

ADR-002: El Evento incluye pagos, contrato y cambios dentro de su límite de consistencia

Título	Frontera del Aggregate Evento
Estado	Propuesta
Contexto	El evento solo se confirma con contrato y anticipo del 30% (BR01) y no puede cerrarse con saldo pendiente (BR09, RNF06)
Alternativas	Aggregate CuentaDelEvento separado, sincronizado por eventos de dominio
Decisión	Pagos, contrato y solicitudes de cambio viven dentro del Evento
Consecuencias	Beneficio: los invariantes INV-E1 e INV-E5 se cumplen de inmediato. Costo: Aggregate más grande y riesgo de conflictos de concurrencia. Mitigación: bloqueo optimista y mantener fuera Propuesta y Solicitudes de Proveedor
Trazabilidad	BR01, BR09, BR10, RNF01, RNF06, US03, US14, US15

ADR-003: Alcance transaccional de la Unit of Work y publicación de eventos

Campo	Contenido
Título	Una transacción por Aggregate, con dos excepciones documentadas
Estado	Propuesta
Contexto	RNF08 exige que una modificación aceptada actualice las partes relacionadas sin versiones contradictorias; BR07 exige información vigente única
Alternativas	Estrictamente un Aggregate por transacción con consistencia eventual entre Evento y Propuesta
Decisión	Una transacción por Aggregate, salvo el alta inicial (CU-01) y el cambio de evento (CU-04), donde Evento y Propuesta se confirman juntos. Los Domain Events se publican después del *commit*
Consecuencias	Beneficio: no hay estados contradictorios entre cambio y propuesta. Costo: transacciones más amplias en dos casos y menos escalabilidad. Trabajo posterior: si la carga crece, evaluar consistencia eventual
Trazabilidad	RNF08, BR07, BR10, US07, US08, US09, US10

ADR-004: CQRS básico sin bases separadas ni bus de mensajes

Campo	Contenido
Título	CQRS de nivel básico
Estado	Propuesta
Contexto	Hay lecturas con forma distinta a la escritura: propuesta vigente, plazos de proveedores, saldo y rentabilidad (RNF02, BN05, BN06, BN07, US12)
Alternativas	Un solo modelo para leer y escribir; CQRS completo con almacenes y mensajería
Decisión	Commands atendidos por Application Services; Queries atendidas por manejadores de lectura con View Models; mismo almacén de datos

Campo	Contenido
Consecuencias	Beneficio: lecturas simples sin cargar Aggregates. Costo: más clases y dos caminos para mantener. No se adopta event sourcing ni buses por no justificarse en el caso
Trazabilidad	RNF02, BN05, BN06, BN07, US12

ADR-005: Resolución provisional de las ambigüedades H1 a H5

Campo	Contenido
Título	Reglas provisionales para contradicciones de la Unidad 2
Estado	Propuesta (pendiente de validación)
Contexto	Contradicciones sobre la liquidación, el borde de 15 días, el cargo automático, su base de cálculo y el cobro de último minuto (sección 1.4)
Alternativas	Esperar la validación antes de diseñar; fijar cada regla de forma rígida en el dominio
Decisión	Borde de 15 días inclusivo (BR03); cargo calculado automáticamente con autorización de Administración; base del cargo sobre el incremento; liquidación permitida en "Confirmado" o "Finalizado"; servicio adicional sin pago no se marca como prestado
Consecuencias	Beneficio: el diseño avanza sin inventar reglas nuevas. Costo: puede requerir cambios cuando se validen. Las reglas viven en PoliticaDeCambios y en el Evento, por lo que un ajuste es local
Trazabilidad	BR03, BR04, BR05, BR08, BR09, US09, US10, US13, US14

10. Matriz de trazabilidad

ID	Necesidad o regla (BRD)	Historia	Caso de uso	Componente de diseño	Decisión
T01	BN02, BR07, RNF02	US01, US07	CU-01	Propuesta, VersionDePropuesta, ConsultarPropuestaVigente	ADR-001, ADR-004
T02	BR10, RNF01	US07	CU-01, CU-04	SolicitudDeCambio (condición anterior y nueva), versiones inmutables	ADR-002
T03	BN01, BR07	US02	CU-02	Propuesta.aceptar, ContratacionService.aceptar_propuesta	ADR-003
T04	BR01, BN07	US03	CU-02	Evento.registrar_anticipo, INV-E1, Pago	ADR-002
T05	RNF03	US03	CU-02	Validación del rol Administración en Evento (Responsable)	ADR-002
T06	BN03, BN06	US04	CU-03	ProveedoresService.solicitar_disponibilidad, SolicitudAProveedorEnviada, NotificadorAProveedores	ADR-001, ADR-003
T07	BR02, OB02	US05	CU-03	SolicitudDeProveedor (INV-S1)	ADR-001
T08	BR02, BN06	US06	CU-03	expirar_solicitudes_vencidas, SolicitudDeProveedorExpirada, AlertasACoordinacion	ADR-003
T09	BN03, RNF07, BR07	US08	CU-04	CambioAplicado, NotificadorAProveedores	ADR-003
T10	BR03	US09	CU-04	PoliticaDeCambios, ReglaDeCargoPorUrgencia	ADR-005
T11	BR04, BN04, BN05	US10	CU-04	SolicitudDeCambio pendiente, autorizar_cambio (Administración), cargo del 15%	ADR-005
T12	RNF08	US07, US09	CU-04	Unidad de Trabajo con Evento y Propuesta	ADR-003
T13	BR06, BN08	US11	Extensión	SolicitudDeProveedor (sustitución), Proveedor.es_backup	ADR-001
T14	BN05, OB03	US12	Consulta	ConsultarRentabilidadPorEvento	ADR-004
T15	BR09, RNF06	US14, US15	Extensión	Evento (INV-E5, liquidación y cierre)	ADR-002, ADR-005
T16	BR05, BR08	US13	Extensión	ServicioAdicional dentro del Evento	ADR-005

11. Validación (Etapa 9)

Validación	Pregunta	Resultado propuesto (el equipo debe confirmarlo)
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	Sí: la matriz de la sección 10 relaciona cada componente con BN, BR, RNF, historia y ADR
Lenguaje	¿Los mismos términos significan lo mismo en BRD, UML, código y ADR?	Sí: los nombres salen del Glosario; se agregaron tres términos (sección 3.1)
Responsabilidad	¿Las reglas permanecen en el dominio?	Sí: 30%, 48 h, 15 días y cierre viven en Evento, SolicitudDeProveedor y PoliticaDeCambios; los servicios solo coordinan
Consistencia	¿Clases y secuencias representan la misma arquitectura?	Sí: los participantes de las secuencias son clases o puertos del diagrama de clases (sección 8)
Persistencia	¿El dominio evita depender de SQL u ORM?	Sí: solo conoce interfaces; la infraestructura se describe aparte (sección 5.3)
Transacción	¿El límite de la operación y su manejo de error están definidos?	Sí: sección 5.2, con *rollback* ante cualquier error
IA	¿Cada aporte de IA fue revisado y registrado?	Pendiente: el equipo debe completar la bitácora (sección 13)

Lista final de comprobación pendiente para el equipo: empresa y alcance de la Unidad 2 mantenidos; patrón respaldado con evidencia; ADR y matriz entregados; diagramas fuente y exportaciones; repositorio con README; y que todos puedan defender las decisiones.

12. Conclusiones

La arquitectura propuesta protege las reglas que el BRD marca como críticas (anticipo, plazos, límite de 15 días, trazabilidad de cambios y cierre sin saldo pendiente) separando el dominio, la aplicación y la infraestructura. El Domain Model se justifica por las reglas combinadas, los estados y los invariantes del caso; los servicios coordinan sin absorber reglas; los repositorios y la Unidad de Trabajo aíslan la persistencia; y CQRS básico separa las consultas sin sobredimensionar la solución.

Quedan cinco ambigüedades de la Unidad 2 (H1 a H5) resueltas de forma provisional, que deben validarse antes de considerar cerrado el diseño.

Consecuencia negativa aceptada de la decisión principal: el Aggregate Evento es más grande para garantizar consistencia inmediata, lo que aumenta el riesgo de conflictos de concurrencia y la complejidad de su mapeo (ADR-002).

13. Bitácora de uso de IA

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
1	Preparar el caso: revisar los productos de la Unidad 2 y proponer casos de uso	Se adjuntaron la guía de la actividad, el BRD, el glosario, las historias de usuario y el BPMN, y se pidió ayuda con la Actividad 3	Cuatro casos de uso con actores, reglas e historias; cinco hallazgos H1 a H5 sobre la Unidad 2	La IA propuso el recorte del proceso principal y las decisiones provisionales de los hallazgos como supuestos	El equipo aceptó los cuatro casos de uso propuestos
2	Redactar la parte teórica de las etapas 2 a 9	Se pidió hacer todo lo que indica la Actividad 3 y después solo la parte teórica	Borrador de este informe: patrón, dominio, servicios, repositorios, CQRS, SOLID, especificación UML, ADR, trazabilidad y validación	Supuestos H1 a H5; los hallazgos SOLID describen un borrador inicial de diseño propuesto por la IA	Pendiente: el equipo debe revisar y registrar qué se conserva, qué se corrige y por qué
3	Revisar la arquitectura con principios SOLID y diseñar ADRs.	"Analiza la propuesta actual contra los principios SOLID, señala violaciones y genera los borradores de los ADRs principales."	Detección de acoplamiento en notificaciones (violación de ISP) y planteamiento de ADR-001 a ADR-005.	La IA sugirió aplicar Event Sourcing y bases de datos de lectura/escrituras separadas para CQRS	Se rechazó Event Sourcing por sobreingeniería (ADR-004) y se dividió la interfaz de notificación en tres interfaces pequeñas (ISP).
4	Representación UML y validación de trazabilidad.	"Genera la especificación de componentes para el Diagrama de Clases y los Diagramas de Secuencia (CU-02 y CU-04), asegurando trazabilidad total."	Estructura detallada de participantes y secuencia de mensajes para flujos normales y excepciones de cambios tardíos.	Asumió que las versiones de propuesta anteriores se sobreescribían al aceptar un cambio.	Se corrigió reafirmando la invariante INV-P1: las versiones son strictly inmutables y no se sobreescriben.

14. Preguntas para la defensa: guía de respuesta

Respuestas de apoyo para ensayar; cada integrante debe poder explicarlas con sus propias palabras.

Pregunta	Respuesta guía
¿Qué evidencia del negocio influyó más en la selección del patrón?	Las reglas numéricas y de tiempo combinadas (30%, 48 h, 15 días: BR01 a BR04) junto con los estados del evento, la propuesta y el proveedor, y los invariantes de BR07 y BR09. Es lógica con estado, no solo registros
¿Qué responsabilidad fue la más difícil de ubicar y cómo tomaron la decisión?	La regla de los 15 días y el cargo del 15%: parecía de servicio o de Evento. Se decidió una política en el dominio (PoliticaDeCambios) porque es una regla del negocio que puede cambiar (ADR-005)
¿Qué regla protege el diseño y qué ocurriría si estuviera en una capa incorrecta?	INV-E1: no confirmar sin contrato y 30%. En el servicio podría omitirse desde otro punto de entrada (otra pantalla o un proceso por lote) y se confirmarían eventos sin anticipo
¿Qué cambió después de la revisión con SOLID?	Se movió la regla de 15 días al dominio, se cambiaron subclases de Proveedor por atributos, se dividió la interfaz de notificaciones y se introdujeron Reloj y puertos (sección 7)
¿Qué propuesta de la IA rechazaron o corrigieron y por qué?	El equipo debe responder con su bitácora real. Candidatas de este borrador: las subclases de Proveedor, el cálculo del cargo en el servicio y los supuestos H1 a H5 que requieren validación
¿Qué consecuencia negativa aceptaron al tomar su decisión arquitectural principal?	Un Aggregate Evento grande y más estructura inicial que un CRUD, a cambio de consistencia inmediata en los invariantes (ADR-001 y ADR-002)